

Table of contents

Chapter 0 – Tools

- 0.1. Code, functions and commands
- 0.2. Compilation
- 0.3. Build details
- 0.4. The GNU C Compiler
 - 0.4.1 The linker
 - 0.4.2 Executable files and cross-compiling
- 0.5. The standard library and the headers
- 0.6. Make
- 0.7. Autoconf
- 0.8. Gdb
- 0.9. The Linux File System
- 0.10. Documentation
 - 0.10.1 `gman` and `gtkman`
 - 0.10.2 `whatis`
 - 0.10.3 `info`
 - 0.10.4 `apropos`
 - 0.10.5 The *glibc* documentation
- 0.11. The source code
- 0.12. Error handling and exit functions
- 0.13. The Standards
- 0.14. The GNU General Public License
- 0.15. Theory and insights
 - 0.15.1 The GNU C Compiler: `gcc`
 - 0.15.2 The GNU Debugger: `gdb`
 - 0.15.3 Bytes order
 - 0.15.4 Static and dynamic linking
- List of tables
- List of figures

Chapter I – File I/O

- I.1 Reading files
 - I.1.1 Opening files: `open` and `close`
 - I.1.2 Reading from file: `read`
 - I.1.3 Writing to file: `write`
 - I.1.4 The standard library functions: `fopen`, `fclose` and `fcloseall`
 - I.1.5 Reading one character at a time: `fgetc` and `fputc`
 - I.1.6 Reading one line at a time: `fgets` and `fputs`
- I.2 Opening files
 - I.2.1 The `fdopen` and `fileno` functions
 - I.2.2 The `freopen` function
 - I.2.3 The open flags
- I.3 Writing files

- I.4 Setting file position
 - I.4.1 The `lseek` function
 - I.4.2 Getting and setting the position: `fgetpos` and `fsetpos`
 - I.4.3 The `fseeko` and `ftello` functions
- I.5 Appending to files
- I.6 Creating files
 - I.6.1 The open `O_EXCL` flag
- I.7 Truncating files
- I.8 Reading and writing at a given offset
- I.9 Duplicating a file descriptor
 - I.9.1 The `dup2` function
 - I.9.2 The `dup3` function
 - I.9.3 Setting the `O_CLOEXEC` flag
 - I.9.4 `dup` errors
- I.10 Synchronizing the data
 - I.10.1 The `sync` function
 - I.10.2 Synchronization errors
- I.11 File flags
 - I.11.1 Duplicating a file descriptor
 - I.11.2 File descriptor flags
 - I.11.3 File status flags
 - I.11.4 File descriptors and open file description
- I.12 Multiplexed I/O
 - I.12.1 The `poll` function
 - I.12.2 The `ppoll` function
 - I.12.3 The `epoll` interface
- I.13 Binary I/O
- I.14 Error handling
 - I.14.1 The `errno` macro
 - I.14.2 The `strerror*` functions
 - I.14.3 The *end-of-file* and error indicators
 - I.14.4 The `err` and `warn` functions
 - I.14.5 The *glibc* error reporting functions
 - I.14.6 The `program_invocation_name` variable
- I.15 Buffered I/O
 - I.15.1 Buffering types
 - I.15.2 The interface to `stdio` `FILE` structure
 - I.15.3 The `fwide` function
- I.16 Memory streams
- I.17 Formatted I/O
 - I.17.1 `printf` and `scanf`
 - I.17.2 Other `printf*` functions
- I.18 System calls and library functions
- I.19 Scatter-Gather I/O
 - I.19.1 The `readv` and `writev` functions
 - I.19.2 The `preadv` and `pwritev` functions
- I.20 Unreading

- I.21 Formatted I/O with variable number of arguments
 - I.21.1 `vprintf`
 - I.21.2 `vscanf`
 - I.21.3 `sscanf`
- I.22 Temporary files
 - I.22.1 The `mkstemp*` family functions
 - I.22.2 The `tmpfile` function
 - I.22.3 Temporary files with `open`
 - I.22.4 Temporary directories: `mkdtemp`
- I.23 Advices
- I.24 Asynchronous I/O
 - I.24.1 POSIX AIO
 - I.24.2 Multiple I/O requests in one call
 - I.24.3 The `aio_init` function of the `real-time` library
 - I.24.4 Linux AIO – `io_uring`
 - I.24.5 The `liburing` library
 - I.24.6 Copying a file with `liburing`
 - I.24.7 The `io_uring` system calls
- I.25 File locking
 - I.25.1 Advisory record locking
 - I.25.2 Opening file description locks
 - I.25.3 The locks on the system
 - I.25.4 The `flock` function
 - I.25.5 The `lockf` function
- I.26 Stream locking
 - I.26.1 Locking `FILE` for `stdio`
 - I.26.2 Nonlocking `stdio` functions
 - I.26.3 Nonlocking functions for *wide-characters*
- I.27 Pipes and FIFO
 - I.27.1 The `pipe` function
 - I.27.2 The `popen` function
 - I.27.3 Two-way communication
 - I.27.4 FIFOs
 - I.27.6 Pipes tunable values
 - I.27.7 I/O on pipes and FIFOs
 - I.27.8 Pipes and formatted I/O
 - I.27.9. The `splice` and `vmsplice` functions
 - I.27.10 Duplicating pipe data
- I.28 `fcntl` commands
 - I.28.1 Managing signals
 - I.28.2 Leases
 - I.28.3 File and directory change notification: `dnotify`
 - I.28.4 Changing the capacity of a pipe
 - I.28.5 File Sealing
 - I.28.6 File read/write hints
- I.29 File allocation
- I.30 Theory and insights
 - I.30.1 I/O

- I.30.2 File descriptors
- I.30.3 File sizes data types
- I.30.4 Positioning data types
- I.30.5 Multibyte representation conversion
- I.30.6 Functions for *wide characters*
- I.30.7 Reading with `sscanf`
- I.30.8 Writing to memory streams
- I.30.9 The `getopt` function
- I.30.10 Synchronous and Asynchronous I/O
- I.30.11 The `aiomonitor.c` program
- I.30.12 The `io_uring_sqe` structure
- I.30.13 The functions of `liburing`
- I.30.14 The program from the `io_uring` manual page
- I.30.15 Pipes and `O_ASYNC`
- I.30.16 Feature Test Macros
- I.30.17 Copying files with `copy_file_range`
- I.30.18 Copying files with `sendfile`

List of tables

List of figures

Chapter II – Files and directories

II.1 Files

- II.1.1 Stat of files: `stat`, `fstat` and `lstat`
- II.1.2 File permissions
- II.1.3 Process credentials and privileges
- II.1.4 The file mode creation mask: `umask`
- II.1.5 Checking file accessibility: `access` and `faccessat`
- II.1.6 Changing file permissions: `chmod` and `fchmod`
- II.1.7 Changing file ownership: `chown`, `fchown` and `lchown`
- II.1.8 Creating and removing links: `link`, `linkat` and `unlink`
- II.1.9 Renaming files and directories: `rename` and `renameat`
- II.1.10 Symbolic links: `symlink` and `readlink`
- II.1.11 File times: `utimensat`, `futimens`, `utime`, `utimes`, `futimes`,
`lutimes`
- II.1.12 I-node flags: `ioctl`
- II.1.13 Immutable files
- II.1.14 Extended attributes: `setxattr`, `getxattr`, `removexattr`,
`listxattr`

II.2 Directories

- II.2.1 Creating and removing directories: `mkdir`, `mkdirat` and `rmdir`
- II.2.2 Open and reading directories: `opendir`, `fdopendir` and `readdir`
- II.2.3 Closing directories: `closedir`
- II.2.4 Scanning a directory: `scandir` and `scandirat`
- II.2.5 Seeking a directory: `telldir`, `seekdir` and `rewinddir`
- II.2.6 Directory file descriptor: `dirfd`
- II.2.7 File tree walk: `nftw`

- II.2.8 Current working directory: `getcwd`, `chdir` and `fcwd`
- II.2.9 Process root directory: `chroot`
- II.3 Pathname and files configuration
 - II.3.1 Resolving paths and links: `realpath` and `canonicalize_file_name`
 - II.3.2 Working with pathnames: `dirname` and `basename`
 - II.3.3 Pathnames and filenames configuration: `fpathconf` and `pathconf`
 - II.3.4 Getting configuration at runtime with `sysconf`
 - II.3.5 Getting configuration dependent string variables: `confstr`
 - II.3.6 Getting configuration from the CLI with `getconf`
 - II.3.7 Summary
- II.4 Device files
 - II.4.1 Major and minor numbers: `makedev`, `major` and `minor`
 - II.4.2 Creating a device file: `mknod` and `mknodat`
 - II.4.3 Creating FIFO: `mkfifo` and `mkfifoat`
 - II.4.4 The random number generator: `getrandom` and `getentropy`
- II.5 Access control list
- II.6 Monitoring file events
 - II.6.1 The *inotify* interface
 - II.6.2 The *fanotify* interface
- II.7 Capabilities
 - II.7.1 File capabilities
 - II.7.2 The *Bounding Set*
 - II.7.3 The *Securebits* flags
 - II.7.4 Listing the contents of the capabilities sets
 - II.7.5 Capabilities data types
 - II.7.6 Child capabilities
- II.8 The Linux file system
 - II.8.1 The *ext4* layout
 - II.8.2 Mounting a filesystem: `mount` and `umount`
 - II.8.3 *ext4* filesystem operations
 - II.8.4 The *swap* area: `swapon` and `swapoff`
 - II.8.5 The `proc` file system
 - II.8.6 The `tmpfs` file system
 - II.8.7 The `sysfs` file system
 - II.8.8 Getting filesystem statistics : `statfs` and `statvfs`
- II.9 Quotas and resource limits
 - II.9.1 Managing quotas
 - II.9.2 Resource limits: `getrlimit`, `setrlimit` and `prlimit`
- II.10 Theory and insights
 - II.10.1 Getting file permissions with `stat`
 - II.10.2 Setting *umask*
 - II.10.3 Unlinking a file while a process has an open descriptor
 - II.10.4 Scanning directories with filters
 - II.10.5 The `openat` and `openat2` functions
 - II.10.6 `name_to_handle` and `open_by_handle_at`
 - II.10.7 Extended stat of files: `statx`
 - II.10.8 Changing root directory

- II.10.9 *Inotify*
- II.10.10 *Inotify* and *Fanotify*
- II.10.11 *ext4* File attributes
- II.10.12 *ext4* Journaling
- II.10.13 Mount namespaces
- II.10.14 The `unshare` function and the *namespaces*
- II.10.15 The `setns` function
- II.10.16 Changing the properties of a mount
- II.10.17 ID-mapped mounts
- II.10.18 Mount information
- II.10.19 New mount API
- II.10.20 The *Virtual File System*

List of tables

List of figures

Chapter III – Environment and system databases

III.1 Time functions

- III.1.1 Getting the time: `time`, `gettimeofday` and `settimeofday`
- III.1.2 Setting time with `clock_gettime` and `clock_settime`
- III.1.3 Clocks on Linux
- III.1.4 Setting a new time: `tm`
- III.1.5 Time conversion functions
- III.1.6 Formatting date and time: `strftime` and `strptime`
- III.1.7 Time conversion: `getdate`

III.2 The environment

- III.2.1 Working with environment variables: `getenv` and `setenv`
- III.2.2 Environment during an `execve`

III.3 Locale

- III.3.1 Setting locale: `setlocale`
- III.3.2 Working with locale: `localeconv`, `newlocale` and `uselocale`
- III.3.3 Duplicating a locale object: `duplocale`

III.4 Users database: `/etc/passwd`

- III.4.1 Users and groups files
- III.4.2 The `/etc/passwd` file
- III.4.3 Working with the users database : `getpwent` and `setpwent`
- III.4.4 Working with records : `fgetpwent` and `putpwent`

III.5 Groups database: `/etc/group`

- III.5.1 Reading the groups database: `getgrnam` and `getgrgid`
- III.5.2 Working with the database: `getgrent` and `setgrent`
- III.5.3 Getting the groups list: `getgrouplist`
- III.5.4 Supplementary groups: `getgroups`, `setgroups` and `initgroups`
- III.5.5 Real and effective group ID: `getgid`, `getegid`, `setgid`, `setegid`
- III.5.6 Saved Set-User-Id and Set-Group-Id
- III.5.7 Checking if a process is in a group: `group_member`

III.6 Users functions

- III.6.1 Setting user ID: `getuid`, `geteuid`, `setuid` and `seteuid`

- III.6.2 `getresgid` and `getresuid`
- III.7 Password database: `/etc/shadow`
 - III.7.1 Creating encrypted passphrase: `crypt`
 - III.7.2 Generating a salt: `crypt_gensalt`
 - III.7.3 Hiding the input with BSD `readpassphrase`
- III.8 Login
 - III.8.1 Working with the `utmp` file: `getutent`, `getutid`, `setutent`
 - III.8.2 The login: `login` and `logout`
- III.9 Hosts, networks, protocols, services databases
 - III.9.1 The hosts database: `/etc/hosts`
 - III.9.2 The networks database: `/etc/networks`
 - III.9.3 The protocols database: `/etc/protocols`
 - III.9.4 The services database: `/etc/services`
- III.10 System identification
 - III.10.1 Getting system information: `uname`
 - III.10.2 Getting and setting hostname: `gethostname` and `sethostent`
- III.11 Pluggable Authentication Modules
 - III.11.1 PAM Authentication: `pam_authenticate`
 - III.11.2 PAM Session: `pam_open_session` and `pam_close_session`
 - III.11.3 Process environment and PAM environment: `pam_getenv`, `pam_putenv`
 - III.11.4 PAM items: `pam_get_item` and `pam_set_item`
 - III.11.5 Changing password: `pam_chauthtok`
- III.12 Theory and insights
 - III.12.1 The Process Execution Environment
 - III.12.2 The `intmax_t` and `uintmax_t` data types
 - III.12.3 Using `initgroups` to set up the supplementary groups
 - III.12.4 Command line tools for the groups
 - III.12.5 The *Name Service Switch*
 - III.12.6 Command line tools for the users
 - III.12.7 The login process
 - III.12.8 PAM functions summary
- List of tables
- List of figures

Chapter IV – Memory

- IV.1 Executable files
 - IV.1.1 The *Executable and Linkable Format* (ELF)
 - IV.1.2 The ELF Header
 - IV.1.3 Section and Segments
 - IV.1.4 The symbol table
 - IV.1.5 The Relocation table
 - IV.1.6 The Dynamic section
 - IV.1.7 The Note section
 - IV.1.8 Reading ELF
 - IV.1.9 Loading ELF into memory
- IV.2 Start and end of a program

- IV.2.1 The `main` function
- IV.2.2 The `_exit` and `_Exit` functions
- IV.2.3 Normal process termination
- IV.2.4 `at_exit` and `on_exit`
- IV.2.5 Process termination and exit status
- IV.2.6 Aborting a program: `abort`
- IV.3 Command line arguments
 - IV.3.1 Parsing options with `getopt`
 - IV.3.2 Parsing options with `getopt_long`
 - IV.3.3 The `getsubopt` function
 - IV.3.2 The `argp` interface
- IV.4 Memory allocation
 - IV.4.1 Memory allocation functions
 - IV.4.2 Processes and memory
 - IV.4.3 The GNU Allocator
 - IV.4.4 The `malloc_trim` function
 - IV.4.5 The `alloca` function
 - IV.4.6 Allocating aligned memory
 - IV.4.7 The `getpagesize` function
- IV.5 Memory mapped I/O
 - IV.5.1 The `mmap` function
 - IV.5.2 Private file mapping
 - IV.5.3 Shared file mapping
 - IV.5.4 Anonymous mapping
 - IV.5.5 Extending a mapped file
 - IV.5.6 Remapping memory
- IV.6 Memory functions
 - IV.6.1 The `memcpy` function
 - IV.6.2 The `memmove` function
 - IV.6.3 Memory initialization: `memset`
 - IV.6.4 Memory search: `memchr` and `memmem`
 - IV.6.5 Comparing memory: `memcmp`
 - IV.6.6 Obfuscating memory: `memfrob`
- IV.7 Protect the memory
 - IV.7.1 The `mprotect` function
 - IV.7.2 Memory protection keys
- IV.8 Locking memory
 - IV.8.1 The `mlock` function
 - IV.8.2 The `mincore` function
- IV.9 RAM-Backed Files and Secret Memory
 - IV.9.1 Anonymous RAM-backed file: `memfd_create`
 - IV.9.2 Private anonymous RAM-backed memory: `memfd_secret`
- IV.10 Memory advice
 - IV.10.1 The `madvise` function
 - IV.10.2 The `posix_madvise` function
 - IV.10.3 Remote memory advice: `process_madvise`
- IV.11 Nonlocal gotos

- IV.11.1 The `setjmp` and `longjmp` functions
- IV.11.2 The `sigsetjmp` and `siglongjmp` functions
- IV.12 Core dump file
- IV.13 Dynamically loaded objects
 - IV.13.1 Opening a shared object: `dlopen`
 - IV.13.2 Obtaining the address of a symbol: `dlsym`
 - IV.13.4 Obtaining information about a loaded object: `dlinfo`
 - IV.13.5 Translating address to symbolic information: `dladdr`
 - IV.13.6 Walking through a list of shared objects: `dl_iterate_phdr`
- IV.14 Theory and insights
 - IV.14.1 The dynamic linker
 - IV.14.2 The C Runtime
 - IV.14.3 Memory allocation parameters
 - IV.14.4 `malloc` related functions
 - IV.14.5 Heap consistency checking
 - IV.14.6 Mapping to a specific address
 - IV.14.7 Memory-management terms glossary
 - IV.14.8 Getting information about physical pages: `get_phys_pages`
 - IV.14.9 Getting the number of processors: `get_nprocs`
 - IV.14.10 Querying the cache: `cachestat`
 - IV.14.11 Non-Uniform Memory Architecture: `numa`
- List of tables
- List of figures

Chapter V – Processes

- V.1 Processes
 - V.1.1 Getting the process identifier: `getpid` and `getppid`
 - V.1.2 Creating a process: `fork`
 - V.1.3 Creating a process blocking the parent: `vfork`
 - V.1.4 Blocking execution: `wait`, `waitpid` and `waitid`
 - V.1.5 The `wait3` and `wait4` functions
 - V.1.6 Process file descriptors
 - V.1.7 Creating child processes: `clone` and `clone3`
 - V.1.8 Comparing processes: `kcmp`
 - V.1.9 Executing a shell command: `system`
 - V.1.10 Executing a program: `execve`
 - V.1.11 Executing a program via file descriptor: `fexecve`
 - V.1.12 The `exec` family functions
 - V.1.13 Controlling a process: `prctl`
 - V.1.14 Process data transfer: `process_vm_read`
 - V.1.15 Secure Computing state of the process: `seccomp`
 - V.1.16 Secure Computing state of the process: `libseccomp`
 - V.1.17 Seccomp user-space notification mechanism: `seccomp_unotify`
 - V.1.18 Seccomp user-space notification mechanism with `libseccomp`
 - V.1.19 Process accounting: `acct`
 - V.1.20 Process scheduling: `nice` and `setpriority`

V.2 Process groups, sessions and terminal

V.2.1 Process groups: `getpgid` and `setpgid`

V.2.2 Sessions: `setsid` and `getsid`

V.2.3 Controlling terminal: `tcsetpgrp`

V.2.4 Job control

V.2.5 Identifying the controlling terminal: `ctermid` and `ttyname`

V.2.6 Orphaned process groups

V.3 Signals

V.3.1 Signal overview

V.3.2 Signal details

V.3.3 Describing signals: `strsignal` and `psignal`

V.3.4 Signal handling: `sigaction`

V.3.5 Signal set operations: `sigemptyset` and `sigfillset`

V.3.6 Signal handlers

V.3.7 Sending signals to the caller: `raise`

V.3.8 Sending signals to another process: `kill` and `killpg`

V.3.9 Sending signals to a specific thread: `tgkill` and `pthread_kill`

V.3.10 Sending real-time signals: `sigqueue` and `pthread_sigqueue`

V.3.11 Setting alarm clock: `alarm`

V.3.12 Blocking signals: `sigprocmask`

V.3.13 Checking for pending signals: `sigpending`

V.3.14 Handling signals with nonlocal jumps

V.3.15 Waiting for signals: `pause` and `sigsuspend`

V.3.16 Waiting for signals: `sigwait` and `sigwaitinfo`

V.3.17 Signal file descriptor: `signalfd`

V.3.18 Alternate signal stack: `sigaltstack`

V.3.19 Sleeping: `sleep`, `nanosleep`, `usleep`, `clock_nanosleep`

V.4 Timers

V.4.1 Interval timers: `getitimer` and `setitimer`

V.4.2 POSIX timers: `timer_create`

V.4.3 Arming and disarming timers: `timer_gettime` and `timer_settime`

V.4.4 Overruns and deletion: `timer_getoverrun` and `timer_delete`

V.4.5 CPU-time clocks: `clock_getcpuclockid`, `pthread_getcpuclockid`

V.5 Threads

V.5.1 POSIX threads: `pthread`

V.5.2 Thread ID: `gettid`, `pthread_self` and `pthread_equal`

V.5.3 Creating threads: `pthread_create` and `pthread_join`

V.5.4 Thread attributes: `pthread_attr_init` and `pthread_attr_destroy`

V.5.5 Terminating threads: `pthread_exit` and `pthread_cancel`

V.5.6 Clean-up handlers: `pthread_cleanup_push`, `pthread_cleanup_pop`

V.5.7 Sending signal to threads: `pthread_kill` and `pthread_sigmask`

V.5.8 Thread-specific data: `pthread_key_create`

V.5.9 Mutexes

V.5.10 Mutex attributes

V.5.11 Timed mutex locking: `pthread_mutex_timedlock`

V.5.12 Read-Write Locks

V.5.13 Condition variables

- V.5.14 Spin locks
- V.5.15 Barriers
- V.5.16 Fork handlers: `pthread_atfork`
- V.5.17 Semaphores
- V.5.18 Futexes
- V.5.19 Thread name: `pthread_setname_np`
- V.6 Daemon processes and logging
 - V.6.1 Daemon overview
 - V.6.2 The `daemon` function
 - V.6.3 The `systemd` service API: `sd-daemon.h`
 - V.6.4 Logging: `syslog`
 - V.6.5 Logging: `sd-journal`
- V.7 Theory and insights
 - V.7.1 Values for `flags` and `clone_args.flags`
 - V.7.2 `prctl` operations
 - V.7.3 The `seccomp_unotify(2)` manual page program
 - V.7.4 CPU sets macros
 - V.7.5 Linux control groups: `cgroup`
 - V.7.6 Tracing a process: `ptrace`
 - V.7.7 Process self-debugging: `backtrace`
 - V.7.8 Thread attributes functions
 - V.7.9 Patterns of synchronization mechanisms
- List of tables
- List of figures

Chapter VI – Interprocess Communication and Terminals

- VI.1 Introduction
- VI.2 POSIX Message Queues
 - VI.2.1 Creating a message queue: `mq_open`, `mq_close`, `mq_unlink`
 - VI.2.2 Sending and receiving messages: `mq_send` and `mq_receive`
 - VI.2.3 Message queue attributes: `mq_getattr` and `mq_setattr`
 - VI.2.4 Message queue notification: `mq_notify`
 - VI.2.5 Notification via thread
 - VI.2.6 Notification via signal
- VI.3 POSIX Named Semaphores
 - VI.3.1 Creating and deleting semaphores: `sem_open`, `sem_close`, `sem_unlink`
 - VI.3.2 Process-local mutual exclusion
 - VI.3.3 Interprocess mutual exclusion
 - VI.3.4 Two-way synchronization pattern
- VI.4. POSIX Shared Memory
 - VI.4.1 Overview
 - VI.4.2 Producer-consumer pattern
 - VI.4.3 Client-server pattern
- VI.5 Sockets
 - VI.5.1 Overview
 - VI.5.2 Creating sockets: `socket` and `shutdown`

- VI.5.3 Byte order conversion functions
- VI.5.4 Addressing: `getaddrinfo` and `getnameinfo`
- VI.5.5 Address formats: `inet_ntop` and `inet_pton`
- VI.5.6 Setting the socket address: `bind`
- VI.5.7 Receiving connections: `listen` and `accept`
- VI.5.8 Connecting to sockets: `connect`
- VI.5.9 Socket information: `getsockname`, `getpeername`, `getsockopt`
- VI.5.10 Data transmission: `send` and `recv`
- VI.5.11 Connection-oriented client-server uptime service
- VI.5.12 Non-blocking sockets and asynchronous I/O
- VI.5.13 Out-of-band data: `socketatmark`
- VI.5.14 Sending and receiving multiple messages: `sendmmsg` and `recvmmsg`

VI.6 UNIX Domain Sockets

- VI.6.1 Unnamed UNIX domain sockets: `socketpair`
- VI.6.2 Named UNIX domain sockets: `sockaddr_un`
- VI.6.3 Datagram sockets: `SOCK_DGRAM`
- VI.6.4 Stream-oriented sockets and message-oriented sockets
- VI.6.5 Abstract sockets
- VI.6.6 Socket options and ancillary messages: `cmsg`
- VI.6.7 Passing file descriptors within a single process
- VI.6.8 Passing file descriptors between parent and child processes
- VI.6.9 Passing file descriptors and credentials between unrelated processes
- VI.6.10 A secure file access service
- VI.6.11 Sequenced-packet sockets: `SOCK_SEQPACKET`

VI.7 Terminal I/O.

- VI.7.1 Identifying terminal file descriptors: `isatty`
- VI.7.2 Terminal properties: `termios`
- VI.7.3 Retrieving and changing terminal settings: `tcgetattr` and `tcsetattr`
- VI.7.4 Canonical, noncanonical and raw mode
- VI.7.5 Line control
- VI.7.6 Line speed
- VI.7.7 Special characters
- VI.7.8 Window terminal size
- VI.7.9 Command-line utilities
- VI.7.10 `ncurses`

VI.8 Pseudo terminals

- VI.8.1 Overview
- VI.8.2 UNIX 98 pseudoterminal
- VI.8.3 `script` and `expect`
- VI.8.3 Opening pseudoterminal: `getpt` and `posix_openpt`
- VI.8.4 Unlocking pseudoterminal: `grantpt` and `unlockpt`
- VI.8.5 Getting pseudoterminal name: `ptsname`
- VI.8.6 Pseudoterminal pair: `openpty`, `forkpty` and `loginpty`

VI.9 Theory and insights

- VI.9.1 POSIX Message queues limits files
- VI.9.2 System V interprocess communication
- VI.9.3 Network interfaces functions

VI.9.4 Virtual console: `vcs`

VI.9.5 Serial terminal lines: `ttys`

List of tables

List of figures